1. Persiapan Data

Import library yang diperlukan

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
from itertools import product
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix
from google.colab import drive
```

Sambungkan ke Google Drive dan arahkan ke path data yang akan digunakan

```
drive.mount('/content/drive')
```

```
path_data = "/content/drive/MyDrive/Bengkel Coding/Pertemuan 2/Dataset/adult.csv"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Baca dataset

Pada data yang kita gunakan kali ini, tantangannya adalah bahwa data ini masih belum memiliki nama kolom, maka nanti akan kita beri nama pada kolomnya termasuk akan kita analisis mengenai langkah apa saja yang akan kita gunakan dalam melakukan pre-processing data

```
df = pd.read_csv(path_data)
df.head()
```

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United-States	<=50K

Langkah berikutnya: Buat kode dengan df | Lihat plot yang direkomendasikan | New interactive sheet

```
df.tail()
```

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
32555	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support	Wife	White	Female	0	0	38	United-States	<=50K
32556	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	0	0	40	United-States	>50K
32557	58	Private	151910	HS-grad	9	Widowed	Adm-clerical	Unmarried	White	Female	0	0	40	United-States	<=50K
32558	22	Private	201490	HS-grad	9	Never-married	Adm-clerical	Own-child	White	Male	0	0	20	United-States	<=50K
32559	52	Self-emp-inc	287927	HS-grad	9	Married-civ-spouse	Exec-managerial	Wife	White	Female	15024	0	40	United-States	>50K

2. Pre-Processing Data

Memberikan nama kolom pada data

```
col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education_num', 'marital_status',
             'occupation', 'relationship', 'race', 'sex', 'capital_gain', 'capital_loss',
             'hours_per_week', 'native_country', 'income']
```

```
df.columns = col_names
df.head()
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40

Langkah berikutnya: Buat kode dengan df | Lihat plot yang direkomendasikan | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32560 entries, 0 to 32559
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32560 non-null  int64
 1   workclass       32560 non-null  object
 2   fnlwgt          32560 non-null  int64
 3   education       32560 non-null  object
 4   education_num   32560 non-null  int64
 5   marital_status  32560 non-null  object
```

```
 6   occupation      32560 non-null  object
 7   relationship    32560 non-null  object
 8   race            32560 non-null  object
 9   sex             32560 non-null  object
 10  capital_gain    32560 non-null  int64
 11  capital_loss    32560 non-null  int64
 12  hours_per_week  32560 non-null  int64
 13  native_country  32560 non-null  object
 14  income          32560 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

tidak ditemukan data yang hilang karena jumlah seluruh value sama.

```
df.describe()
```

	age	fnlwgt	education_num	capital_gain	capital_loss	hours_per_week
count	32560.000000	3.256000e+04	32560.000000	32560.000000	32560.000000	32560.000000
mean	38.581634	1.897818e+05	10.080590	1077.615172	87.306511	40.437469
std	13.640642	1.055498e+05	2.572709	7385.402999	402.966116	12.347618
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.178315e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.783630e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.370545e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	99.000000

Menghitung nilai unik pada masing-masing class

Di sini kita akan melakukan pengecekan terkait nilai-nilai unik yang ada pada tiap-tiap kolom yang bertipe kategorikal

```
unique_counts = df.nunique()
print(unique_counts)
```

```
age                 73
workclass            9
fnlwgt           21647
education           16
education_num       16
marital_status       7
occupation          15
relationship         6
race                 5
sex                  2
capital_gain       119
capital_loss        92
hours_per_week      94
native_country      42
income               2
dtype: int64
```

di sini kita melakukan pengecekan value terhadap seluruh kolom yang berisi string

```
kolom = ["workclass", "education", "marital_status",    "occupation",   "relationship", "race", "sex", "native_country",    "income"]
for col in kolom:
    print(f"Kolom: {col}")
    print(df[col].unique())
    print("-" * 40)
```

```
Kolom: workclass
[' Self-emp-not-inc' ' Private' ' State-gov' ' Federal-gov' ' Local-gov'
 ' ?' ' Self-emp-inc' ' Without-pay' ' Never-worked']
----------------------------------------
Kolom: education
[' Bachelors' ' HS-grad' ' 11th' ' Masters' ' 9th' ' Some-college'
 ' Assoc-acdm' ' Assoc-voc' ' 7th-8th' ' Doctorate' ' Prof-school'
 ' 5th-6th' ' 10th' ' 1st-4th' ' Preschool' ' 12th']
----------------------------------------
Kolom: marital_status
[' Married-civ-spouse' ' Divorced' ' Married-spouse-absent'
 ' Never-married' ' Separated' ' Married-AF-spouse' ' Widowed']
----------------------------------------
Kolom: occupation
[' Exec-managerial' ' Handlers-cleaners' ' Prof-specialty'
 ' Other-service' ' Adm-clerical' ' Sales' ' Craft-repair'
 ' Transport-moving' ' Farming-fishing' ' Machine-op-inspct'
 ' Tech-support' ' ?' ' Protective-serv' ' Armed-Forces'
 ' Priv-house-serv']
----------------------------------------
Kolom: relationship
[' Husband' ' Not-in-family' ' Wife' ' Own-child' ' Unmarried'
 ' Other-relative']
----------------------------------------
Kolom: race
[' White' ' Black' ' Asian-Pac-Islander' ' Amer-Indian-Eskimo' ' Other']
----------------------------------------
Kolom: sex
[' Male' ' Female']
----------------------------------------
Kolom: native_country
[' United-States' ' Cuba' ' Jamaica' ' India' ' ?' ' Mexico' ' South'
 ' Puerto-Rico' ' Honduras' ' England' ' Canada' ' Germany' ' Iran'
 ' Philippines' ' Italy' ' Poland' ' Columbia' ' Cambodia' ' Thailand'
 ' Ecuador' ' Laos' ' Taiwan' ' Haiti' ' Portugal' ' Dominican-Republic'
 ' El-Salvador' ' France' ' Guatemala' ' China' ' Japan' ' Yugoslavia'
 ' Peru' ' Outlying-US(Guam-USVI-etc)' ' Scotland' ' Trinadad&Tobago'
 ' Greece' ' Nicaragua' ' Vietnam' ' Hong' ' Ireland' ' Hungary'
 ' Holand-Netherlands']
----------------------------------------
Kolom: income
[' <=50K' ' >50K']
----------------------------------------
```

karena ada beberapa record yang memiliki value "?" maka akan kita ganti dengan "NaN" kemudian akan kita ganti dengan data mayoritas, kenapa mayoritas?, akan kami sampaikan pada bagian akhir

```
# bersihkan spasi berlebihan pada seluruh kolom bertipe object (string)
df = df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

# ganti tanda "?" dengan NaN
df.replace("?", np.nan, inplace=True)

# ganti NaN dengan nilai mayoritas (modus) di setiap kolom
```

```
for col in df.columns:
    if df[col].dtype == "object":  # hanya untuk kolom kategorikal
        mode_value = df[col].mode()[0]  # ambil nilai modus (terbanyak)
        df[col].fillna(mode_value, inplace=True)

# menampilkan jumlah data yang masih memiliki NaN (seharusnya 0 jika sudah bersih)
print("Jumlah data yang masih memiliki NaN setelah imputasi:\n", df.isnull().sum())
```

```
Jumlah data yang masih memiliki NaN setelah imputasi:
 age               0
workclass         0
fnlwgt            0
education         0
education_num     0
marital_status    0
occupation        0
relationship      0
race              0
sex               0
capital_gain      0
capital_loss      0
hours_per_week    0
native_country    0
income            0
dtype: int64
<ipython-input-56-d94b5b07d1a7>:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using ar
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always beh

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) i


  df[col].fillna(mode_value, inplace=True)
```

```
kolom = ["workclass", "education", "marital_status",    "occupation",   "relationship", "race", "sex", "native_country",    "income"]
for col in kolom:
    print(f"Kolom: {col}")
    print(df[col].unique())
    print("-" * 40)
```

```
Kolom: workclass
['Self-emp-not-inc' 'Private' 'State-gov' 'Federal-gov' 'Local-gov'
 'Self-emp-inc' 'Without-pay' 'Never-worked']
----------------------------------------
Kolom: education
['Bachelors' 'HS-grad' '11th' 'Masters' '9th' 'Some-college' 'Assoc-acdm'
 'Assoc-voc' '7th-8th' 'Doctorate' 'Prof-school' '5th-6th' '10th'
 '1st-4th' 'Preschool' '12th']
----------------------------------------
Kolom: marital_status
['Married-civ-spouse' 'Divorced' 'Married-spouse-absent' 'Never-married'
 'Separated' 'Married-AF-spouse' 'Widowed']
----------------------------------------
Kolom: occupation
['Exec-managerial' 'Handlers-cleaners' 'Prof-specialty' 'Other-service'
 'Adm-clerical' 'Sales' 'Craft-repair' 'Transport-moving'
 'Farming-fishing' 'Machine-op-inspct' 'Tech-support' 'Protective-serv'
 'Armed-Forces' 'Priv-house-serv']
----------------------------------------
Kolom: relationship
['Husband' 'Not-in-family' 'Wife' 'Own-child' 'Unmarried' 'Other-relative']
----------------------------------------
Kolom: race
['White' 'Black' 'Asian-Pac-Islander' 'Amer-Indian-Eskimo' 'Other']
----------------------------------------
Kolom: sex
['Male' 'Female']
----------------------------------------
Kolom: native_country
['United-States' 'Cuba' 'Jamaica' 'India' 'Mexico' 'South' 'Puerto-Rico'
 'Honduras' 'England' 'Canada' 'Germany' 'Iran' 'Philippines' 'Italy'
 'Poland' 'Columbia' 'Cambodia' 'Thailand' 'Ecuador' 'Laos' 'Taiwan'
 'Haiti' 'Portugal' 'Dominican-Republic' 'El-Salvador' 'France'
 'Guatemala' 'China' 'Japan' 'Yugoslavia' 'Peru'
 'Outlying-US(Guam-USVI-etc)' 'Scotland' 'Trinadad&Tobago' 'Greece'
 'Nicaragua' 'Vietnam' 'Hong' 'Ireland' 'Hungary' 'Holand-Netherlands']
----------------------------------------
Kolom: income
['<=50K' '>50K']
----------------------------------------
```

Karena data dengan tipe kategorikal bentuknya masih berupa object dan belum berupa angka, maka selanjutnya kita perlu untuk mengubah agar tipe dari record yang bertipe object menjadi bertipe angka, kita akan menggunakan label encoder dari pustaka sklearn

```
from sklearn.preprocessing import LabelEncoder
kolom = ["workclass", "education", "marital_status",    "occupation",   "relationship", "race", "sex", "native_country",    "income"]

le = LabelEncoder()

for col in kolom:
    df[col] = le.fit_transform(df[col])
df.head()
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	nat
0	50	5	83311	9	13	2	3	0	4	1	0	0	13	
1	38	3	215646	11	9	0	5	1	4	1	0	0	40	
2	53	3	234721	1	7	2	5	0	2	1	0	0	40	
3	28	3	338409	9	13	2	9	5	2	0	0	0	40	
4	37	3	284582	12	14	2	3	5	4	0	0	0	40	

Langkah berikutnya: Buat kode dengan df | Lihat plot yang direkomendasikan | New interactive sheet

untuk kolom target (income) ketika penghasilan <= 50k maka diubah ke angka 0, dan jika penghasilan > 50k diubah ke angka 1

Melakukan Pengecekan Missing Value

```
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])
```

```
Series([], dtype: int64)
```

```
df.describe()
```

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gai
count	32560.000000	32560.000000	3.256000e+04	32560.000000	32560.000000	32560.000000	32560.000000	32560.000000	32560.000000	32560.000000	32560.00000
mean	38.581634	3.094349	1.897818e+05	10.298249	10.080590	2.611794	6.138943	1.446376	3.665848	0.669195	1077.61517
std	13.640642	1.107094	1.055498e+05	3.870317	2.572709	1.506225	3.972623	1.606794	0.848817	0.470510	7385.40299
min	17.000000	0.000000	1.228500e+04	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000
25%	28.000000	3.000000	1.178315e+05	9.000000	9.000000	2.000000	3.000000	0.000000	4.000000	0.000000	0.00000
50%	37.000000	3.000000	1.783630e+05	11.000000	10.000000	2.000000	6.000000	1.000000	4.000000	1.000000	0.00000
75%	48.000000	3.000000	2.370545e+05	12.000000	12.000000	4.000000	9.000000	3.000000	4.000000	1.000000	0.00000
max	90.000000	7.000000	1.484705e+06	15.000000	16.000000	6.000000	13.000000	5.000000	4.000000	1.000000	99999.00000

3. Pemodelan

di sini kita akan menggunakan model Random Forest, kenapa Random Forest?, karena pada hasil pengujian yang telah saya lakukan, Random Forest yang berhasil mendapatkan akurasi tertinggi dibandingkan dengan model-model lainnya

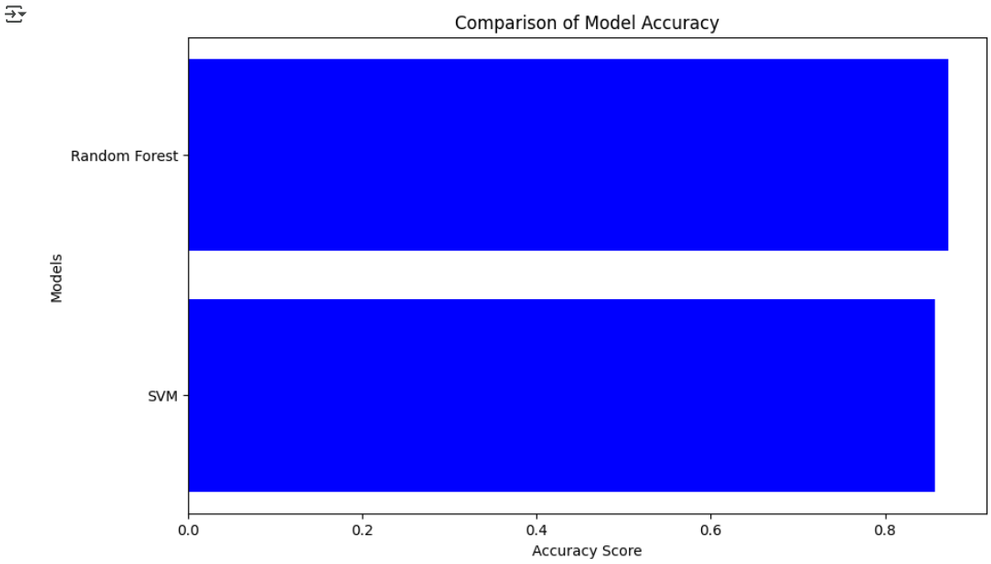
```
# pisahkan fitur dan target
X = df.drop("income", axis=1)
y = df["income"]


# bagi data menjadi train-test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)


# normalisasi fitur numerik
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


# Fungsi evaluasi model
def evaluate_model(model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    report = classification_report(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    return accuracy, precision, recall, f1, report, cm


# handling data yang tidak seimbang
sampling_methods = {
    "No Sampling": None
}


# model yang akan digunakan
models = {
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC()
}


# hyperparameter
param_grid = {
    "Random Forest": {
        "n_estimators": [200],
        "max_depth": [15],
        "min_samples_split": [5],
        "min_samples_leaf": [1],
        "max_features": [None]
    },
    "SVM": {
        "C": [1],
        "kernel": ["rbf"],
        "gamma": ["scale"]
    }
}


# Menyimpan hasil evaluasi
result = {}
conf_matrices = {}
for name, model in models.items():
    best_params = {}
    if name in param_grid and param_grid[name]:
        grid_search = GridSearchCV(model, param_grid[name], cv=5, scoring="accuracy", n_jobs=-1)
        grid_search.fit(X_train, y_train)
        model = grid_search.best_estimator_
        best_params = grid_search.best_params_

    accuracy, precision, recall, f1, report, cm = evaluate_model(model, X_train, X_test, y_train, y_test)
    result[name] = {
        'Best Parameters': best_params,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1 Score': f1,
        'Classification Report': report
    }
    conf_matrices[name] = cm


# Menampilkan hasil terbaik untuk setiap model
for model_name, scores in result.items():
    print(f"Model: {model_name}")
    print(f"Best Parameters: {scores['Best Parameters']}")
    print(f"Accuracy: {scores['Accuracy']}")
    print(f"Precision: {scores['Precision']}")
    print(f"Recall: {scores['Recall']}")
    print(f"F1 Score: {scores['F1 Score']}")
    print(f"Classification Report:\n{scores['Classification Report']}")
    print("=" * 50)
```

```
Model: Random Forest
Best Parameters: {'max_depth': 15, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}
Accuracy: 0.8726965601965602
```

```
Precision: 0.868038348308184
Recall: 0.8726965601965602
F1 Score: 0.8677058763880063
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.95      0.92      4944
           1       0.79      0.64      0.71      1568

    accuracy                           0.87      6512
   macro avg       0.84      0.79      0.81      6512
weighted avg       0.87      0.87      0.87      6512

==================================================
Model: SVM
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.8573402948402948
Precision: 0.8511001135378425
Recall: 0.8573402948402948
F1 Score: 0.8494753684326346
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.95      0.91      4944
           1       0.78      0.57      0.66      1568

    accuracy                           0.86      6512
   macro avg       0.83      0.76      0.78      6512
weighted avg       0.85      0.86      0.85      6512

==================================================
```

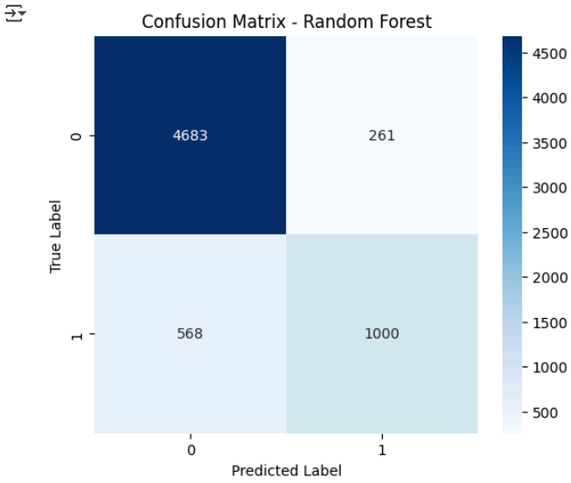
dari hasil report dapat kita lihat bahwa untuk model Random Forest berhasil mendapatkan akurasi sebesar **0.87** atau **87%**, sementara itu untuk model SVM akurasi yang didapatkan yaitu sebesar **0.85** atau **85%**

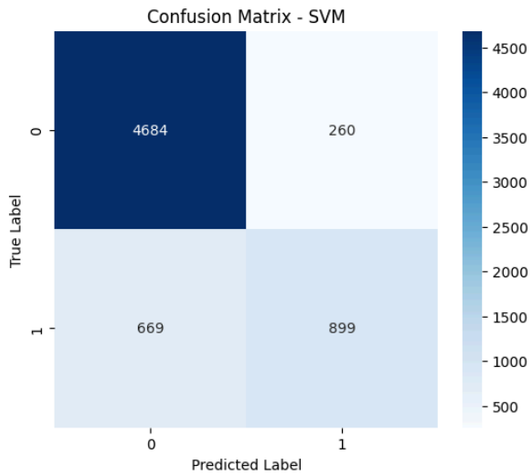
```
# Menampilkan grafik perbandingan akurasi
model_names = list(result.keys())
accuracies = [result[model]['Accuracy'] for model in model_names]

fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(model_names, accuracies, color='blue')
ax.set_xlabel("Accuracy Score")
ax.set_ylabel("Models")
ax.set_title("Comparison of Model Accuracy")
ax.invert_yaxis()

plt.show()
```



pada visualisasi hasil akurasi dapat kita lihat bahwa model Random Forest memiliki akurasi lebih tinggi dibandingkan dengan model SVM, perlu untuk diketahui bahwa akurasi ini kami dapatkan setelah kami melakukan hyperparameter tuning terhadap model-model yang telah kami pilih, sehingga model yang kami gunakan disini adalah model terbaik dari model-model yang terbaik, dan parameter terbaik dari parameter-parameter yang terbaik

```
# Menampilkan confusion matrix untuk setiap model
for model_name, cm in conf_matrices.items():
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title(f"Confusion Matrix - {model_name}")
    plt.show()
```





dari hasil conf matrix di atas dapat kita simpulkan bahwa untuk model Random Forest:

1. terdapat 4683 data yang memiliki class negatif dan terklasifikasi sebagai bagian dari class negatif (**Benar**)
2. terdapat 261 data yang memiliki class negatif namun terklasifikasi sebagai bagian dari class positif (**Salah**)
3. terdapat 1000 data yang memiliki class positif dan terklasifikasi sebagai bagian dari class positif (**Benar**)
4. terdapat 568 data yang memiliki class positif namun terklasifikasi sebagai bagian dari class positif (**Salah**)

---

sementara itu, untuk model SVM: